

一. 线性回归与SVM

1. 分类与回归

分类: 输出离散类别 回归: 输出连续数值

2. 监督学习模型的三个核心设计

假设空间/模型类别 → 预测函数 集合

损失函数 → 衡量误差

优化算法 → 怎样找 loss 最小参数

3. 线性回归: 用线性函数预测连续值

平方损失: $loss = (pred - true)^2$

4. 线性分类、分数与间隔

先计算 $score = w \cdot x$ $\begin{cases} > 0, \text{预测为} +1 \\ < 0, \text{预测为} -1 \end{cases}$

决策边界: $score = 0$

间隔 $margin = score \times y$ (对错、程度)

5. 常见损失

Zero-one $\begin{cases} \checkmark: 0 \\ \times: 1 \end{cases}$

Hinge: $\max\{1 - (w \cdot x), 0\}$

不仅惩罚“分错”, 也惩罚“分得 margin 太小”

logistic: $\log(1 + e^{-w \cdot x})$

逻辑回归使用(解决分类)

6. 梯度下降 GD: 所有样本平均梯度

随机 ~ SGD: 一个样本

Mini-batch GD: 取一个小 batch 作 GD

学习率/步长: 决定每次参数更新走多远

7. 多分类

OVA: 每类一个分类器(该类 ↔ 非该类) (K个)

OvO: 每两类一个分类器 ($\frac{K(K-1)}{2}$ 个)

8. SVM: 使两类样本到边界的最小距离尽可能大

最近边界: 支持向量

x_0 到 $w^T x + b$ 的距离为: $\frac{|w^T x_0 + b|}{\|w\|}$

二. 模型选择和评估

1. 训练集

验证集

测试集

穷举法

调参

评估真实性能

2. 回归任务:

均方误差 $MSE = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$

3. 准确度 ACC: $\frac{\text{正确分类数}}{\text{总数}}$ (类别极不平衡时误导)

错误率 $1 - ACC$

4. 混淆矩阵

预测 \ 真实	正	负
正	TP	FN
负	FP	TN

前面的 T/F 表明是否正确
后面的 P/N 为预测结果

5. 精确率 $Precision = \frac{TP}{TP + FP}$ (预测为正中, 多少实际为正)
关心报出正类是否靠谱

召回率 $Recall = \frac{TP}{TP + FN}$ (实际为正中, 多少预测为正)
关心不能漏掉正类

F1 分数: 调和平均 $\frac{2PR}{P+R}$

6. 过拟合 仅在验证集上表现好 (正则化、早停止)

欠拟合: 能力不足 (再训练)

三. 无监督学习、K-means

训练数据无标签, 只有特征

K-means: 聚类算法 (所得簇仅有编号, 不是事先给的真实)

输入: 点集合 D 希望划分簇数 K

输出: 每个样本属于哪个簇 每簇中心点 (簇心)

步骤: 随机初始化 K 个中心点

↓
计算所有样本到中心点的距离, 分给最近的

重复 → 移动中心点到同类均值处

想优化所有样本到中心的平方距离之和 → 选均值为中心

局限: 必先指定 K 对初始中心敏感 对离群点敏感

适合“团状”簇

四. 深度学习基础

1. 深度学习: 使用多层神经网络, 自动从训练数据中学习特征和分类/预测规则 (使 loss 最小)

2. 一个神经网络层 $\begin{cases} \text{线性变换} \\ \text{非线性激活 (若无, 仍等价于一个线性)} \end{cases}$
从而拟合复杂的非线性关系

交叉熵损失 $H(y, \hat{y}) = - \sum_{i=1}^K y_i \log \hat{y}_i$ $score = -H(y, \hat{y})$

3. Softmax: 多分类输出 $softmax(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$
(先输出 score, 再由 softmax 变为概率)

4. 前向/反向传播

训练循环: 前向(输入, 逐层算) → loss → 反向(算梯度) → 梯度下降更新

5. 对抗样本: 在原始输入上加小扰动, 却可让模型错误预测 → 不“理解”

五. CNN

局部连接: 一个输出神经元只观察感受野

权重共享: 同一个卷积核 (复用)

输入: $x_i \in \mathbb{R}^{H \times W \times C_i}$

一个卷积核: $M \times N \times C_i \rightarrow$ 负责检测一种模式
输出一个通道

更适应图像: 利用局部相关性 (每个神经元只关注局部)

权重共享 (不同位置检测相同模式)

减少参数量、识别不同位置的相似特征

输出大小: $\lfloor \frac{N-K+2P}{S} \rfloor + 1$ (偏置)

参数量: 核数 $\times (K \times K \times \text{通道数} + 1)$

六. Transformer

Self-Attention 允许一个 token 参考所有 token (全局、聚合)

对于 token $\begin{cases} \text{Query} \\ \text{Key} \\ \text{Value} \end{cases} \quad \text{softmax}(\frac{QK^T}{\sqrt{d_k}}) V$

单层可建立远距离依赖 (CNN要叠加)

需要位置信息: token 置换等变, 本身不含序列顺序信息

多头注意力: 从多个不同角度、不同特征子空间同时关注

七. 搜索问题: 输出一串动作 (还要考虑对未来和总代价的影响)

目标: 开始状态 $\rightarrow$ 目标状态

构成: ① 状态/状态空间
 $\rightarrow$ 所有可能状态集合
 $\rightarrow$ 某一具体场景

② 动作 Actions(s)

代价 Cost(s, a)

后继函数 Succ(s, a)

③ 开始状态

④ 结束测试

搜索树: 不断作出动作, 逐层展开找后继

关注“最便宜的路径”

代价一致搜索 UCS: 优先扩展从起点到该状态累计代价最小的状态

状态 $\begin{cases} \text{Explored: 已确定最优路径的状态} \\ \text{Frontier: 已发现, 但未确定最便宜到达方式} \\ \text{Unexplored: 未发现} \end{cases}$ 从 Frontier 中选择

代价非负时可人确保找到最优

不知“离终点还要花多少代价” $\rightarrow$ 浪费

改进: 启发式函数 (估计当前到目标的代价)

$h(s) \approx \text{Future Cost}(s)$

A* 搜索: $f(s) = \text{Past Cost}(s) + h(s)$

优先扩展 f(s) 最小的状态

松弛: 用“去掉限制”构造启发式函数

忽略一部分约束, 以简单问题作为估计

八. 马尔可夫决策过程 (不确定环境下连续决策)

假设: 每个状态只取决于前一个状态和动作

MDP 定义: 包括

States 状态集合, S_{start} , Action(s)

$T(s, a, s')$, $Reward(s, a, s')$, $IsEnd(s)$

γ 折扣因子

目标: 最大期望累计奖励 “随机”

策略 $\pi(s)$: 状态 s 应采取什么行动 (对状态)

回报 $Utility = r_1 + \gamma r_2 + \gamma^2 r_3 \dots$ (一条轨迹)

其价值 Value: $V^\pi(s)$ 从 s 出发遵循 π , Utility 的期望

$V^{opt}(s) = \max_{\pi} V^\pi(s)$: 从 s 出发的最大期望回报

π^{opt} 是 每个 s 都选能实现 $V^{opt}(s)$ 的动作

策略评估: 输入 (MDP, π) 输出 V^π

(给定 π , 计算其在每个状态下的价值)

$V^\pi(s) = \begin{cases} 0, & IsEnd(s) \\ Q^\pi(s, \pi(s)) = \sum_s T(s, a, s') [R(s, a, s') + \gamma V^\pi(s')] \end{cases}$

迭代更新: 所有状态价值初始化为 0

迭代更新直至价值稳定

值迭代: 直接找最优策略

每个状态在全部动作中取最大值

$V^{opt}(s) = \max_{a \in \text{Actions}(s)} \sum_{s'} \dots$

输出: (Q^{opt}, π^{opt})

$\pi^{opt}(s) = \arg \max_a Q^{opt}(s, a)$

九. 强化学习: 不知规则, 交互中学

agent 和环境交互: 将经验 s, a, r, s'

连续交互形成轨迹

基于模型的值迭代: 从数据中得 $\begin{cases} T(s, a, s') \\ R(s, a, s') \end{cases}$

再当作 MDP

无模型强化学习: 直接学 $Q^\pi(s, a)$ 或 $Q^{opt}(s, a)$

① Model-Free Monte Carlo

一条完整轨迹结束后, 用其实际回报作样本

② SARSA (来自经验) (s, a, r, s', a') 自举

$$\hat{Q}^\pi(s, a) \leftarrow (1-\eta) \hat{Q}^\pi(s, a) + \eta [r + \gamma \hat{Q}^\pi(s', a')]$$

估计实际执行策略 π 的 Q 值

③ Q-learning: 学习 Q^{opt}

$$\hat{Q}^{opt}(s, a) \leftarrow (1-\eta) \hat{Q}^{opt}(s, a) + \eta [r + \gamma \max_{a' \in \text{Action}(s')} \hat{Q}^{opt}(s', a')]$$

使用 s' 中 Q 值最大的动作

④ 探索与利用 ϵ -greedy

有概率 ϵ 随机选择动作 $\rightarrow$ 防错过

On-policy: 用当前的策略来改进、评估本身

Off-policy: 用一个策略产生数据, 却学习另一个目标策略